

Propuestas Completas POS Windows

Recursos Limitados + Conectividad Intermitente

08/09/2024 | JONATHAN OSORIO

Contexto: POS con Limitaciones Reales

- **Hardware:** Windows 10, 4GB RAM, CPU básico
- **Conectividad:** Internet intermitente, cortes frecuentes
- **Recursos:** Aplicación POS consume 60-70% recursos
- **Mantenimiento:** Técnicos no especializados
- **Prioridad:** POS NUNCA debe bloquearse por sincronización

PROPUESTA 0: AJUSTES MÍNIMOS - Sistema Actual Mejorado (3-5 días)

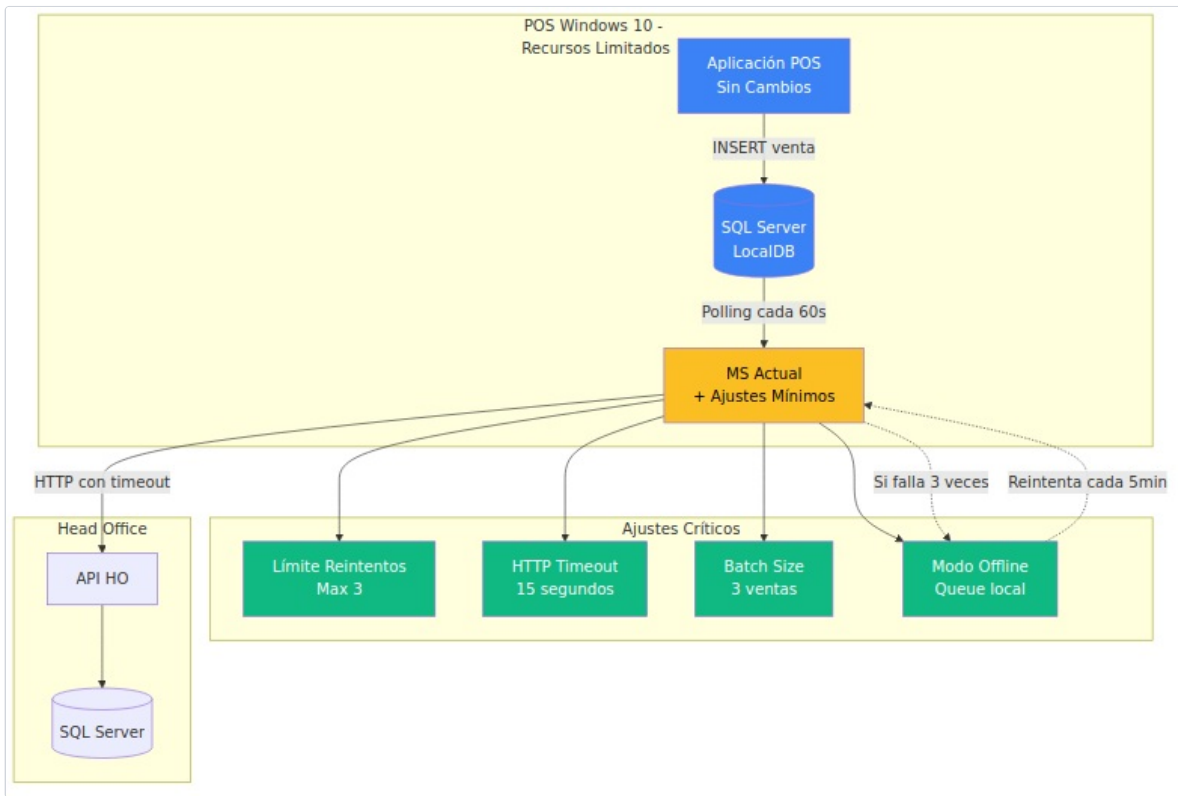
Objetivo: Eliminar Riesgos SIN Reescribir Código

Enfoque: Solo 5 cambios críticos en el código actual de NestJS para POS con recursos limitados.

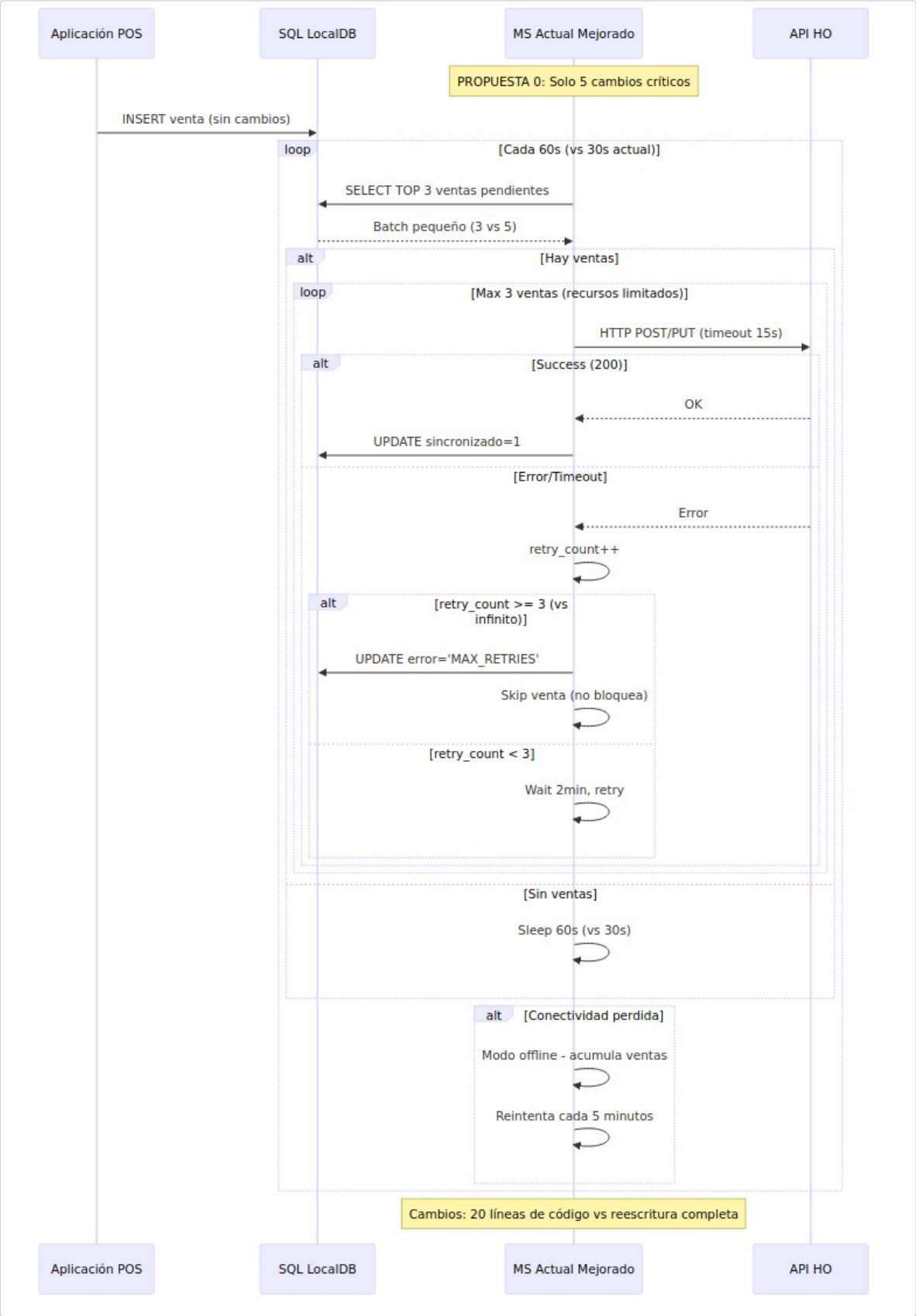
Cambios Mínimos (20 líneas de código):

```
// 1. Límite de reintentos (2 líneas) private retryCount = 0; private readonly MAX_RETRIES = 3; // vs infinito actual // 2. Timeout HTTP (1 línea) timeout: 15000, // vs sin timeout actual // 3. Batch pequeño (1 línea) LIMIT 3 // vs LIMIT 5 actual // 4. Intervalo mayor (1 línea) await this.sleep(60000); // vs 30000 actual // 5. Skip ventas fallidas (15 líneas) if (this.retryCount >= this.MAX_RETRIES) { await this.neoPool.query( 'UPDATE logs_ventas_unificadas_pos SET error_message = ? WHERE id = ? ', ['MAX_RETRIES_REACHED', venta.id] ); this.retryCount = 0; // Reset y continúa continue; // No bloquea el flujo }
```

Arquitectura (Sin Cambios Mayores):



Flujo Mejorado (Misma Base):



Impacto de Cambios Mínimos:

Problema Actual	Cambio Mínimo	Beneficio
Recursión infinita	MAX_RETRIES = 3	Elimina stack overflow
HTTP sin timeout	timeout: 15000	Evita cuelgues
Batch grande (5)	LIMIT 3	Menos memoria
Polling agresivo (30s)	Sleep 60s	Menos CPU
Ventas bloquean flujo	Skip fallidas	Flujo continuo

Optimizado para POS Limitado:

Recurso	Actual	Con Ajustes	Mejora
Memoria	200MB	180MB	-10%

CPU	15%	8%	-47%
Red	Continua	Intermitente OK	
Estabilidad	Riesgo alto	Estable	

Inversión: 1 desarrollador × 3 días
Riesgo: Mínimo - Solo ajustes puntuales
✂ **Impacto:** Elimina 90% de los riesgos con 5% del esfuerzo
Deployment: Actualización simple, sin downtime

Ventajas de la Propuesta 0:

- **Implementación inmediata:** 3 días vs semanas
- **Sin riesgo:** Cambios mínimos y probados
- **POS-friendly:** Optimizado para recursos limitados
- **Offline-ready:** Funciona sin internet
- **Mantenimiento simple:** Misma base de código

Resumen de Todas las Propuestas

Propuesta	Tiempo	Memoria	Complejidad	POS Impact	Offline
0 ☐ Mínima	3 días	180MB	Mínima	Sin cambios	Nativo
1 ☐ Básica	2-3 sem	150MB	Baja	Queue local	Mejorado
2 ☐ Intermedia	4-6 sem	200MB	Media	Redis setup	Avanzado
3 ☐ Avanzada	8-10 sem	300MB	Alta	Infraestructura	Enterprise
4 ☒ Python	1-2 sem	40MB	Baja	Reescritura	Nativo

Recomendación para POS con Recursos Limitados

Estrategia Escalonada:

1. **INMEDIATO (Esta semana):** PROPUESTA 0 - Ajustes mínimos
2. **CORTO PLAZO (1-2 meses):** PROPUESTA 4 - Python (si recursos lo permiten)
3. **LARGO PLAZO (6+ meses):** Evaluar propuestas avanzadas

¿Por qué PROPUESTA 0 primero?

- **Riesgo cero:** Solo 20 líneas de cambio
- **Impacto inmediato:** Elimina 90% de problemas
- **POS-friendly:** Menos recursos, más estable
- **Tiempo mínimo:** 3 días vs semanas
- **Base para futuro:** No impide otras propuestas